



第四届 e B P F 开发者大会

The 4th eBPF Developer Conference (eBPFDC 2026)

eBPF赋能云原生RDMA网络 资源精细化管理

刘明轩
陕西·西安

西北工业大学
2026年4月18日

<https://www.ebpftravel.com>



目录

CONTENTS

01 背景

02 云原生RDMA网络的资源异常

03 eBPF赋能云原生RDMA网络资源管理

04 总结与展望



第四届 e B P F 开发者大会

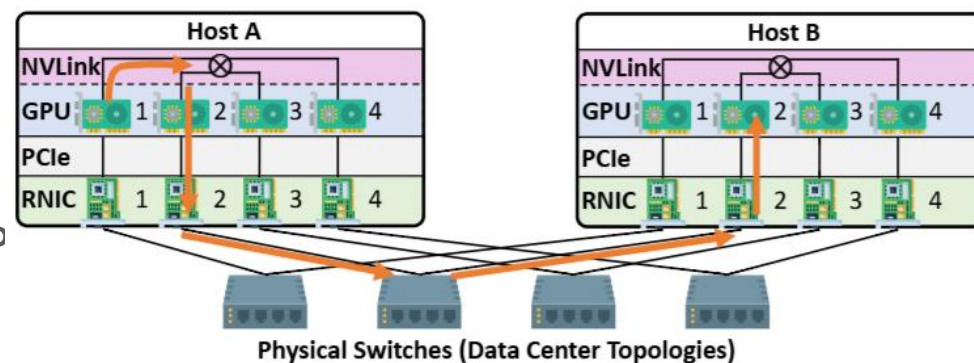
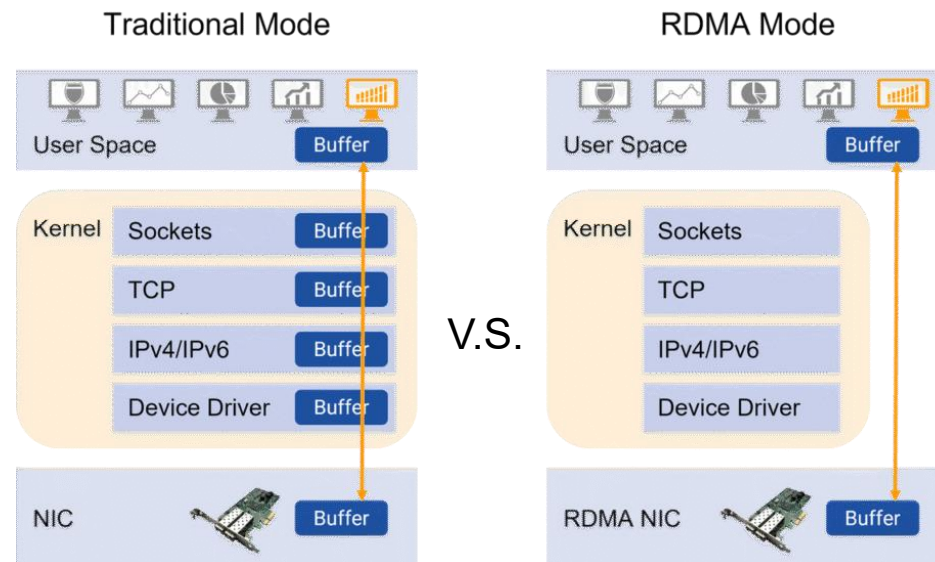
The 4th eBPF Developer Conference (eBPFDC 2026)

01 背景

陕西·西安 2026年4月18日

无处不在的RDMA

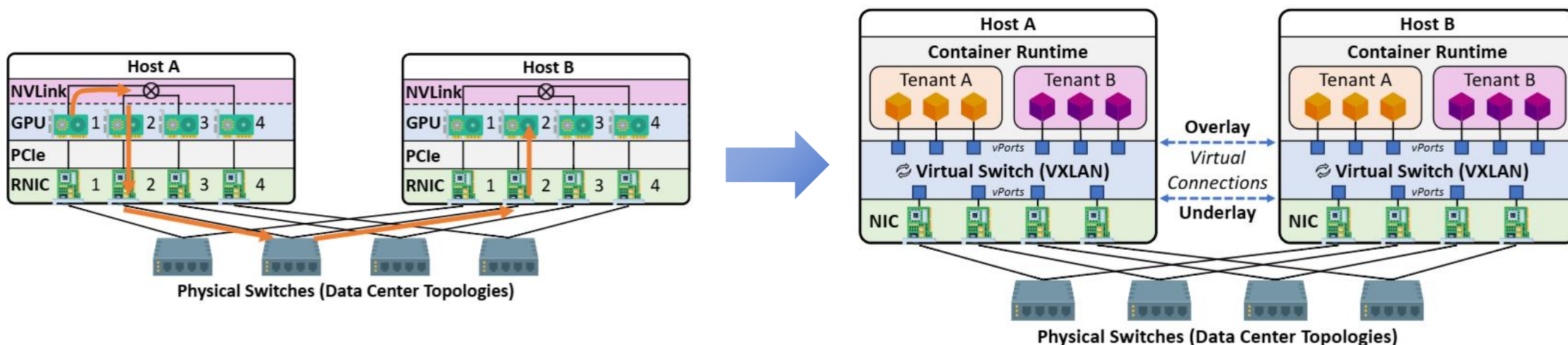
- RDMA: Remote Direct Memory Access
- 传统基础设施中的RDMA:
 - 传统超算: UCX/UCC等
 - 分布式存储中: NVMe-of/Ceph-RDMA等
 - 分布式数据库: Shared-everything DB等
 - ...
- AI基础设施中的RDMA¹: GPUDirect RDMA
 - 大模型训练: NCCL、DeepEP (MoE专用)
 - 大模型推理: NIXL、Mooncake TE等
 - 大模型分发: DeepSeek 3FS、Mooncake P2P
 - ...



1. 图片来源: Liu, Wei, et al. "SkeletonHunter: Diagnosing and Localizing Network Failures in Containerized Large Model Training." Proceedings of the ACM SIGCOMM 2025 Conference. 2025.

云原生RDMA网络

- 云原生：裸机->虚拟机->容器->Serverless->...
- 云厂商售卖RDMA网络资源的方式：与高端实例打包
- 从云服务商的角度看整个RDMA网络^{1,2}
 - 云原生RDMA网络最大的特点：多租户——存在RDMA网络资源的竞争



1. 图片来源: Liu, Wei, et al. "Mitigating Scalability Walls of {RDMA-based} Container Networks." 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25). 2025.
2. 图片来源: Liu, Wei, et al. "SkeletonHunter: Diagnosing and Localizing Network Failures in Containerized Large Model Training." Proceedings of the ACM SIGCOMM 2025 Conference. 2025.



第四届 e B P F 开发者大会

The 4th eBPF Developer Conference (eBPFDC 2026)

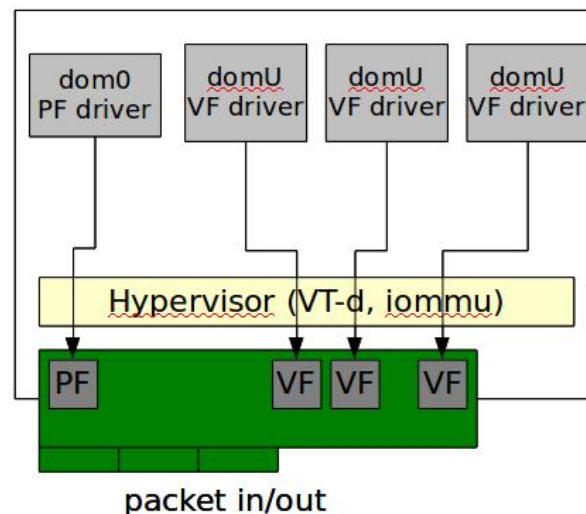
02 云原生RDMA网络的 资源异常

陕西·西安 2026年4月18日



工业上的RDMA网络多租户管理

- 硬件虚拟化：SR-IOV 与 Virtual Function (VF) ¹
- 官方的RDMA QoS硬件方法²:
 - 严格优先级 (Strict Priority)
 - RNIC TC按数字顺序严格优先级 (TC7最高, TC0最低)
 - 速率限制 (Rate Limit)
 - 为每个TC设置最大带宽限制
 - 例如: `mlx_qos -i <interface> -r 3,4,2`
(TC0:3Gbps, TC1:4Gbps, TC2:2Gbps)
 - 增强传输选择 (Enhanced Transmission Selection, ETS)
 - 最小带宽保证, 空闲带宽可被其他TC使用。例如: TC0: 80%保证, TC1: 20%保证。
- 然而, 上述管理方案仅涉及到了“**RDMA流量管理**”, 而非“**RDMA资源管理**”。
- 当恶意租户耗尽RDMA网卡资源时, 传统的RDMA QoS硬件方法能起作用吗? 答案是**不能**

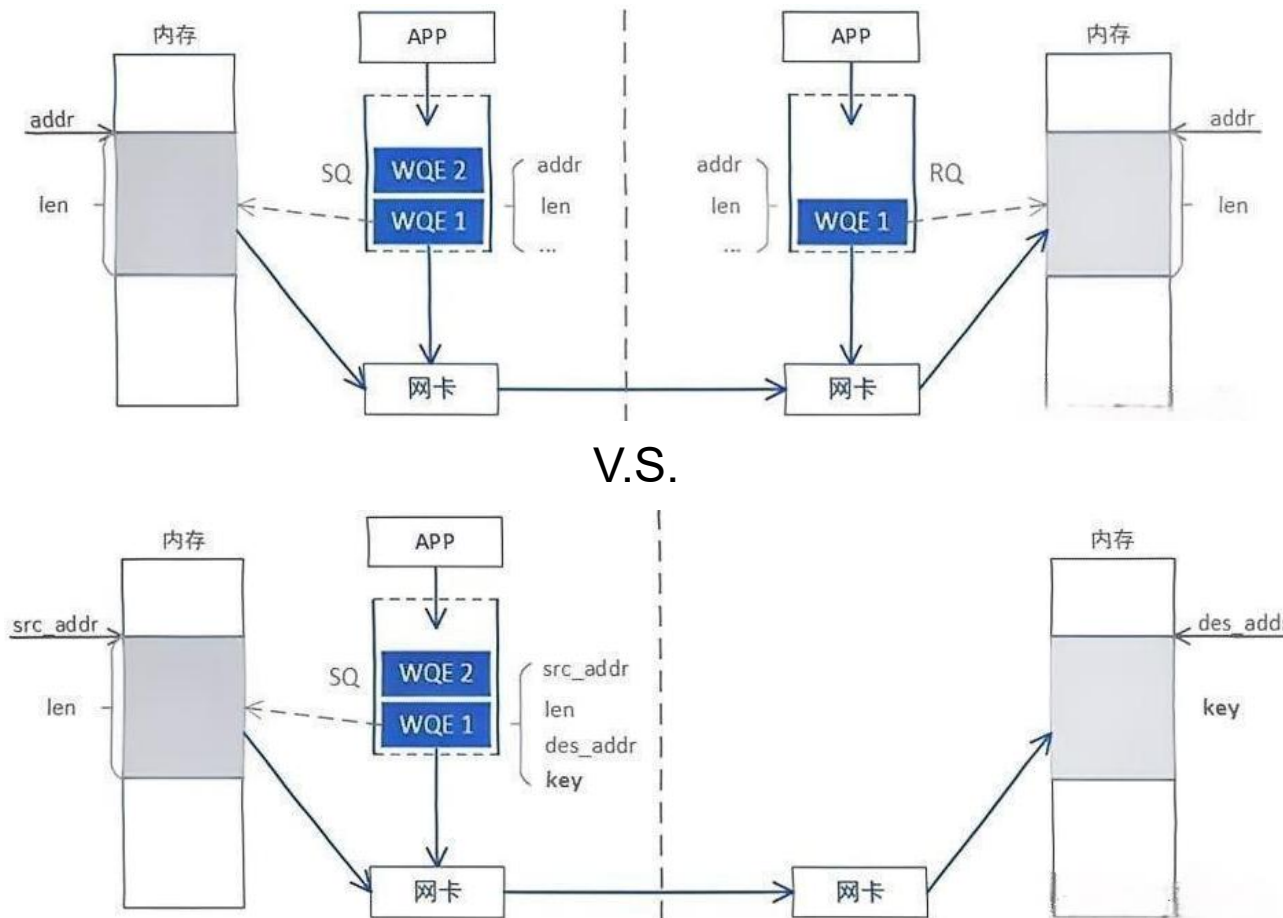


1. 图片来源: http://www.iorchard.net/2016/10/10/sr-iov_on_xen.html

2. NVIDIA MLNX_OFED Documentation. Quality of Service (QoS). [https://docs.nvidia.com/networking/display/mlnxofedv497100lts/quality+of+service+\(qos\)](https://docs.nvidia.com/networking/display/mlnxofedv497100lts/quality+of+service+(qos))

RDMA的资源类型

缩略语	全称
WQ	Work Queue
WQE	Work Queue Entry
QP	Queue Pair
SQ	Send Queue
RQ	Receive Queue
SRQ	Shared Receive Queue
CQ	Completion Queue
CQE	Completion Queue Entry
WR	Work Request
WC	Work Completion
MR	Memory Region
PD	Protection Domain



RDMA Send/Recv
双边操作

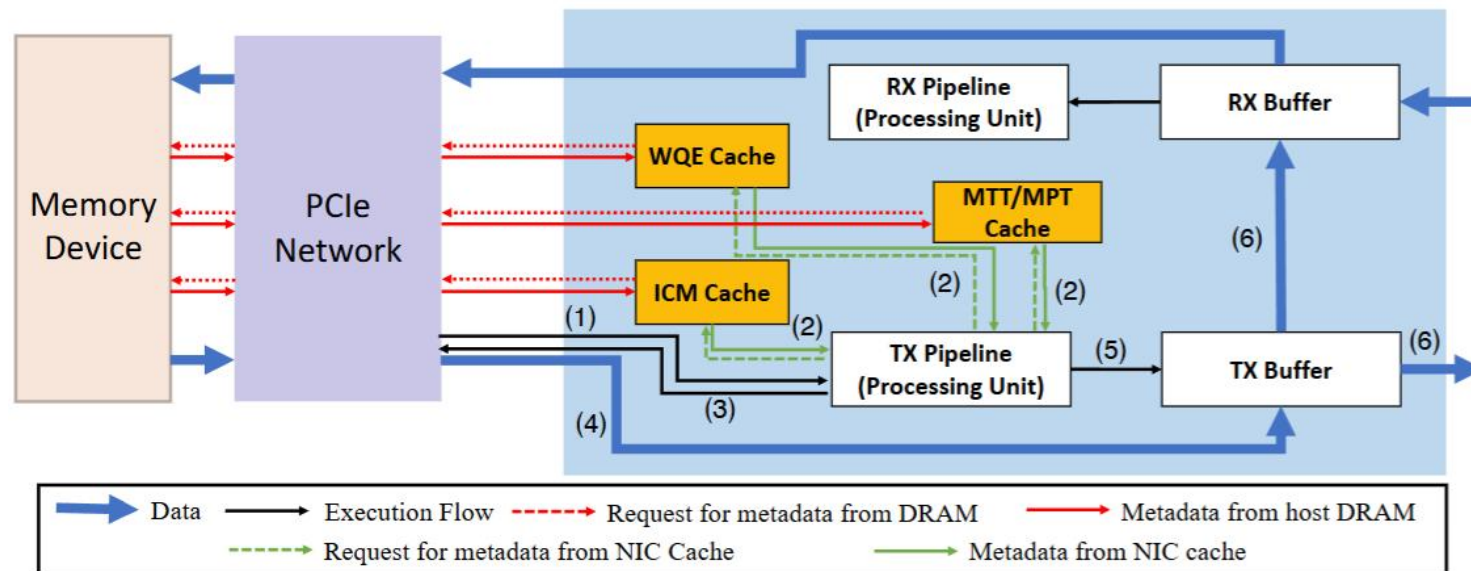
RDMA Write
RDMA Read
单边操作

1. 图片来源: <https://zhuanlan.zhihu.com/p/164908617>

The diagram illustrates the RDMA architecture components and their interactions:

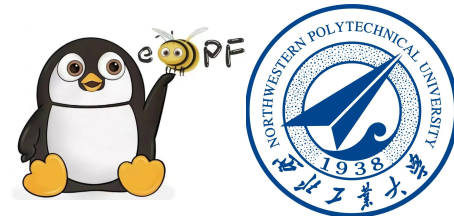
- Host:** Contains three layers:
 - User Application:** Interacts with Userspace Libraries via black arrows (Control Verbs) and red arrows (Data Verbs).
 - Userspace Libraries:** Interacts with the Kernel Driver via black arrows (Control Verbs) and red arrows (Data Verbs).
 - Kernel Driver:** Interacts with the RNIC via black arrows (Control Verbs) and red arrows (Data Verbs).
- RNIC (Network Interface Card):** Contains two processes:
 - Data Verbs Process:** Receives data verbs from the Userspace Libraries and sends them to the Fabric.
 - Control Verbs Process:** Receives control verbs from the Kernel Driver and sends them to the Fabric.
- Fabric:** The external network connecting the RNICs.
- IB Verbs:**
 - Control Verbs:** Represented by black double-headed arrows.
 - Data Verbs:** Represented by red double-headed arrows.
- IB Verbs List:**
 - Control Verbs:** `ibv_open_device`, `ibv_alloc_pd`, `ibv_reg_mr`.
 - Data Verbs:** `ibv_post_send`, `ibv_post_recv`, `ibv_poll_cq`, `ibv_create_cq`, `ibv_create_qp`, `ibv_modify_qp`.

- RDMA Verbs分为Control Verbs和Data Verbs
- RDMA Verbs的处理逻辑大部分都卸载到了RNIC上。



- 当门铃响起时，RNIC首先从主机DRAM通过PCIe获取控制Verbs或数据Verbs请求。
- 两类Processing Unit (TX/RX Pipeline PU) 用来执行控制Verbs任务。
- 三类缓存 (Cache) 缓存部分控制面元数据 (QP Context等)，供PU快速获取。
- 两类队列 (TX/RX Buffer) 用来在数据平面，通过PCIe访问主机DRAM来收发数据。
 - 现有的RDMA QoS硬件方法²，只能影响到两类队列 (TX/RX Buffer)

9



多租户共用RNIC时的资源异常

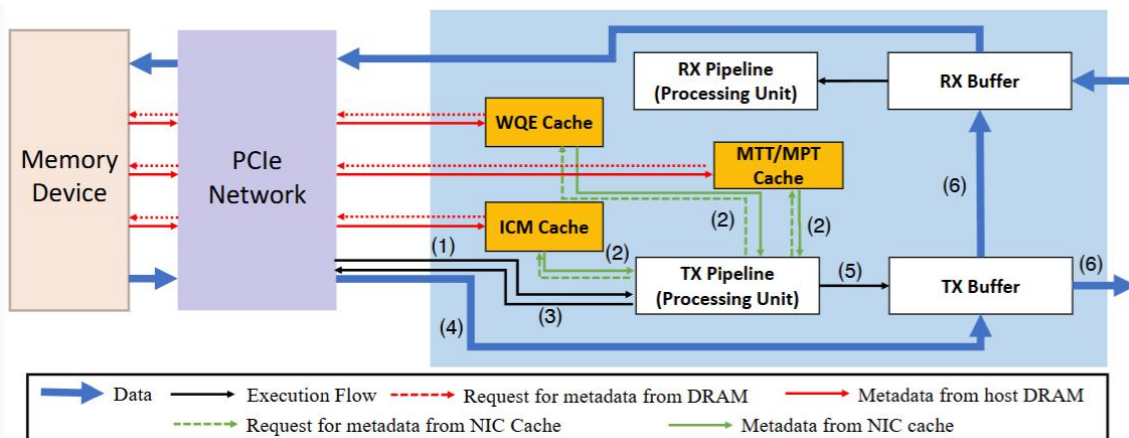
• 相关工作

- Lite@SOSP17
- Husky@NSDI23
- Justitia@NSDI22/PeRF@ATC24
- ScalaCN@NSDI25/SkeletonHunter@SIGCOMM25
- SwiftRDMA@APNet25
- Noisy Neighbor@arXiv25

• 存在的共性问题

- 以故障定位（发现问题）为主，提出的解决方案大部分需要硬件迭代
 - 硬件迭代周期长，受硬件厂商的政策制约
- 隔离方式多处于网卡粒度（基于SR-IOV）或应用粒度（RDMA资源划分）
 - 对RNIC资源的管理相对粗糙

实验量化分析（详见附录）



💡 核心结论

硬件缓存的争用引发数据平面吞吐受损

- ✓ 多租户页表隔离导致有效容量下降70%
- ✓ 高频MR操作触发PCIe事务配额争用
- ✓ 单纯依赖硬件调度无法满足隔离需求

短板效应性能模型-性能取决于三个瓶颈的最小值

- 1 PUI计算瓶颈**
Post-N批处理提升效率
- 2 QPC缓存(ICM Cache)瓶颈**
连接数扩展导致缓存失效
- 3 PCIe事务瓶颈**
控制平面操作挤占数据平面

→ 研究启示

- 需要在操作系统层面实现管控
硬件资源黑盒特性决定了软件层介入的必要性
- 需要立体化的解决方案
从微架构分析→控制平面管控→数据平面流控



第四届 eBPF 开发者大会

The 4th eBPF Developer Conference (eBPFDC 2026)

03 eBPF赋能云原生RDMA 网络资源管理

RESEARCH-ARTICLE | FREE ACCESS |

Wiseswap: Elastic Datacenter Network-Aware Disaggregated Memory for Multi-Tenant Cloud

Authors: [Mingxuan Liu](#), [Jianhua Gu](#), [Tianhai Zhao](#), [Dong Sun](#) | [Authors Info & Claims](#)

WWW '26: Proceedings of the ACM Web Conference 2026 • Pages 5253 - 5262 • <https://doi.org/10.1145/3774904.3792309>

Published: 12 April 2026 [Publication History](#) | Check for updates



陕西 · 西安 2026年4月18日

现有管控机制的不足



Linux Cgroup v2 RDMA子系统

</> 现有资源类型定义

```
RDMACG_RESOURCE_HCA_HANDLE  
RDMACG_RESOURCE_HCA_OBJECT  
// 仅两类粗粒度资源
```

⚠ 管控粒度过粗

无法区分QP、CQ、MR、PD等不同类型的RDMA资源

⚠ 静态配额限制

无法应对突发恶意负载

⚠ 缺乏动态感知

无法识别控制平面风暴攻击

🔍 需要解决的问题

多维度RDMA资源对象限制

QP/CQ/MR/PD独立计数与限制

层次化资源统计

容器视角的资源可见性

动态负载感知

实时监测控制面操作频率

💡 设计目标

细粒度管控

QP、CQ、MR、PD四类核心资源独立限制

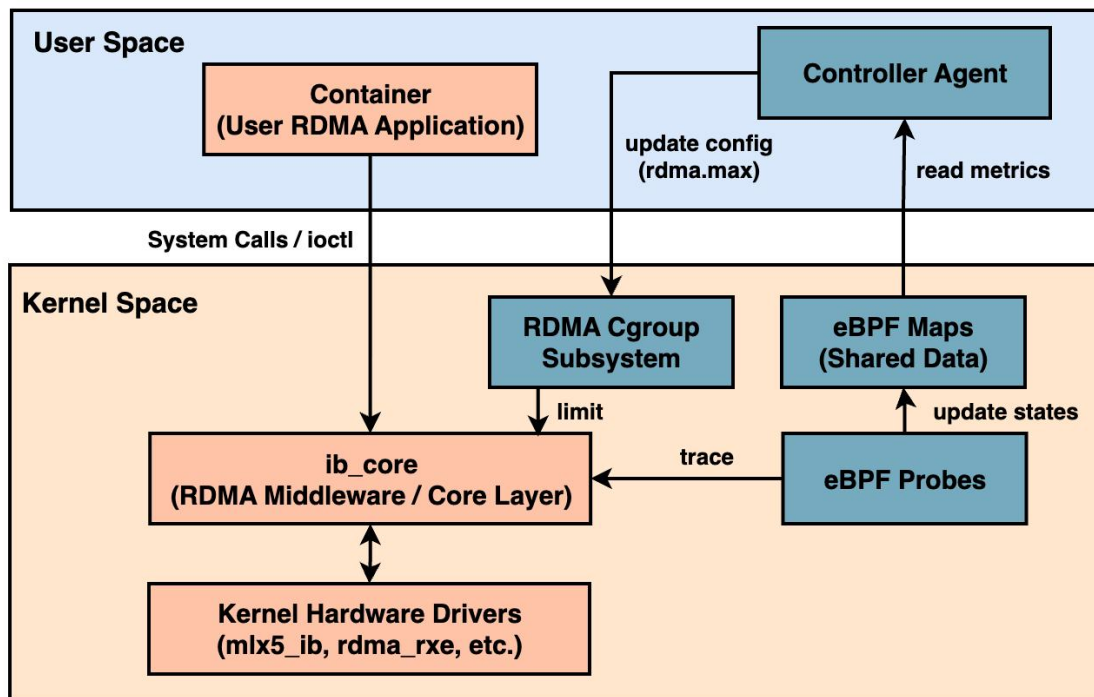
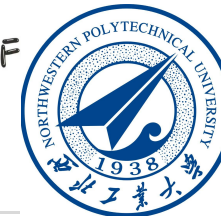
动态防御

毫秒级识别与抑制攻击

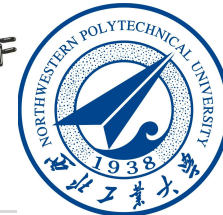
生态兼容

与K8s等编排系统兼容

基于Cgroup+eBPF的动态管理系统架构



基于Cgroup+eBPF的动态管理系统架构



机制与策略分离



执行层 Actuator

- 增强型RDMA Cgroup
- 内核子系统硬限流
- 资源准入控制



感知层 Sensor

- eBPF全景遥测
- API调用频率捕获
- 低开销监控



决策层 Controller

- 用户态智能代理
- AIMD动态调整
- 策略决策引擎

✓ 执行层特点

- 扩展Linux rdmacg_resource_pool
- QP/CQ/MR/PD独立限制
- 内核路径拦截

✓ 感知层特点

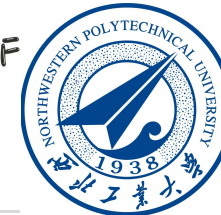
- BPF_PROG_TYPE_KPROBE
- QP→cgroup_id动态映射
- 时间窗口频率统计

✓ 决策层特点

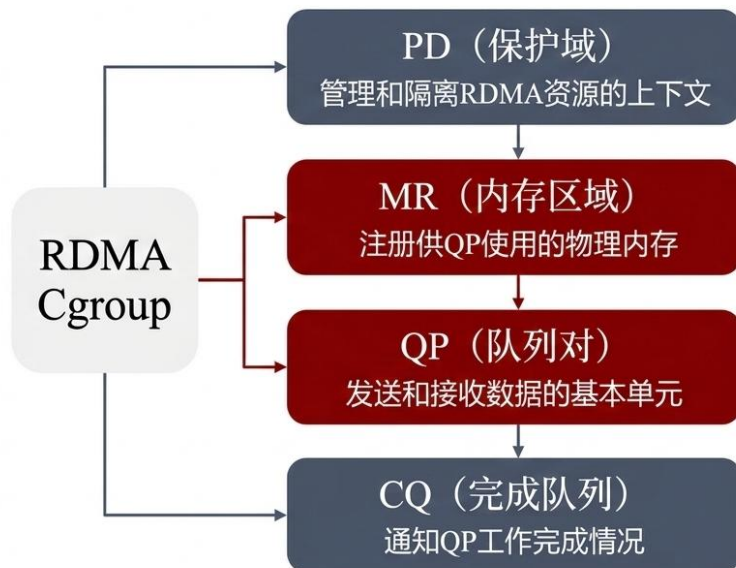
- 100ms周期采样
- AIMD（加性增/乘性减）算法
- 毫秒级响应

闭环控制：感知层采集 → 决策层分析 → 执行层调整 → 持续监控

执行层：增强型Cgroup扩展设计



RDMA Cgroup 新增资源类型



原有资源类型
2类

扩展后资源类型
2+4类

核心数据结构

`rdma_cgroup`
每个Cgroup节点对应一个应用实例

`rdmacg_resource_pool`
为每个RDMA设备维护资源池

`rdmacg_resource`
每种资源的计数与限制

</> 用户态接口

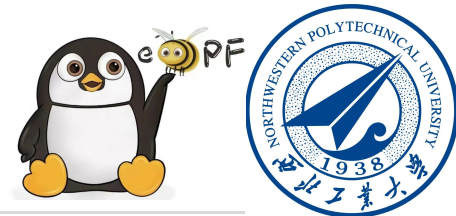
配置接口

```
echo "qp=100 cq=150" > rdma.max
```

状态读取

```
cat rdma.current
```

感知层： eBPF全景遥测



eBPF程序挂载点与功能

挂载类型

BPF_PROG_TYPE_KPROBE
挂载在ib_core核心函数入口

1 资源归属动态映射

BPF Hash Map建立QP→cgroup_id映射

2 控制面负载监测

时间窗口统计操作调用频率

3 低开销采集

Per-CPU Array Map避免锁竞争

</> eBPF程序示例

```
SEC("kprobe/ib_uverbs_reg_mr")
int trace_reg_mr(struct pt_regs *ctx) {
    u32 cgid = get_current_cgroup_id();
    u64 *val = bpf_map_lookup_elem(
        &map_cgroup_ops, &cgid);
    if (val) {
        __sync_fetch_and_add(val, 1);
    }
    return 0;
}
```

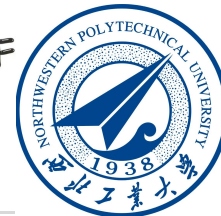
 原子递增操作计数，实现无锁统计

性能优势

~0
数据面开销

< 11%
控制面延迟增加

决策层：AIMD动态调度算法



算法核心思想

↓ 乘性减 (Multiplicative Decrease)
检测到异常时，配额减半

↑ 加性增 (Additive Increase)
负载正常时，线性恢复配额

动态阈值计算

$$Th_critical = \min(\max(Limit_static \times \gamma, C_floor), C_ceil)$$

γ : 突发容忍因子 (1.0)

C_floor : 启动豁免下限 (20)

C_ceil : 物理饱和上限 (500)

算法执行流程



防御效果

毫秒级识别控制平面风暴攻击

公平性

保障正常业务资源不被挤占

灵活性

多个参数调节算法效果

实验验证：功能与开销评估

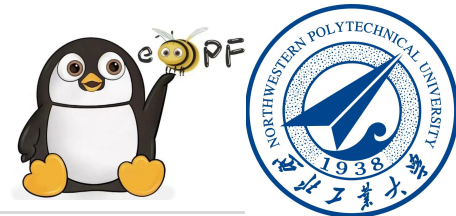
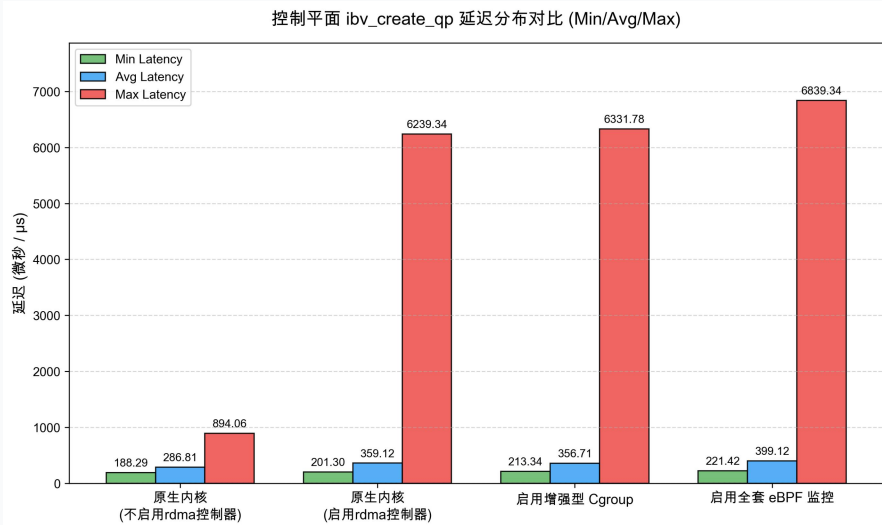


表 4-3 Cgroup 资源硬限制边界测试结果

资源类型 (Resource)	配置上限 (N)	尝试申请 (N + Δ)	实际获批 (Allocated)	边界行为判定 (Result)
Queue Pairs (QP)	100	120	100	✓拒绝第 101 次请求
Completion Queues (CQ)	50	60	50	✓拒绝第 51 次请求
Memory Regions (MR)	200	250	200	✓拒绝第 201 次请求
Protection Domains (PD)	10	20	10	✓拒绝第 11 次请求



原生内核延迟
~359 μs

启用扩展Cgroup+eBPF后延迟
~399 μs

✓ 功能验证结果

资源限制功能

精确在阈值时触发检查，返回-ENOMEM

数据面零侵入

端到端延迟波动在测量误差范围内

📊 关键数据

控制面延迟增加

很少

数据面影响

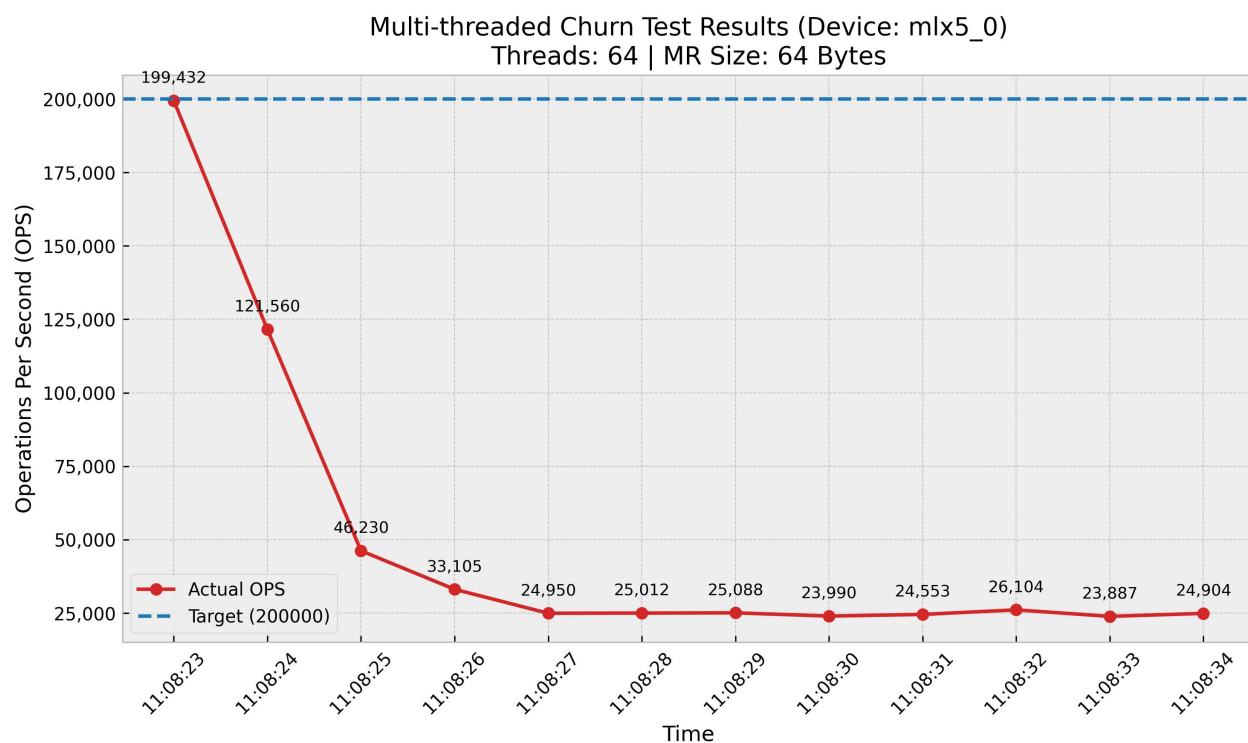
~0%

📌 对低频控制面操作完全可接受

实验验证：动态防御测试



攻击者MR的配额利用率变化



实验设置

攻击者：以200000Hz 频率执行MR注册/销毁

参数：Th_warn=0.8, Cfloor=25000, Cceil=100000,
Limitmin=150

实验结果

攻击启动

T=11:08:23（触发预警）：攻击启动，瞬时请求率飙升至~199,432 OPS，远超阈值

乘性减生效

配额从100降至50→25→12在 3 秒内，系统迅速将实际执行的OPS 从 ~121,560 压缩至 ~33,105，极大缓解了 RNIC 压力。

持续锁定

攻击成功频率稳定在最低配额附近（~25,000），全程未出现反弹。

方案对比



方案优势总结

优于现有的静态资源方案，提供细粒度的4项独立管控与动态防御。

✓ 与现有方案对比

vs Linux Cgroup v2

从2类粗粒度资源扩展到4类细粒度资源

vs 静态配额

引入AIMD动态调整，毫秒级响应

vs 硬件方案

软件层实现，无需修改硬件

★ 核心贡献

- 1 细粒度资源管控
QP/CQ/MR/PD独立限制
- 2 动态防御能力
AIMD算法毫秒级识别攻击
- 3 极低系统开销
数据面零侵入，控制面11%
- 4 生态兼容
扩展Cgroups与K8s等编排系统兼容

Liu, Mingxuan, et al. "Wiseswap: Elastic Datacenter Network-Aware Disaggregated Memory for Multi-Tenant Cloud." *Proceedings of the ACM Web Conference (WWW) 2026*. (CCF-A)





第四届 e B P F 开发者大会

The 4th eBPF Developer Conference (eBPFDC 2026)

04 总结与展望

陕西·西安 2026年4月18日



展望：云原生AI Agent的新蓝海

- 云原生AI Agent场景下的异常定位
 - 基本问题：安全容器/快速启动
 - 增大GPU复用（Multiplexing）密度：构造GPU OS服务更多租户
 - 多租户托管复杂种类Agentic Workflow：如何高效观测？
- eBPF与RDMA的潜力

RDMA特性	Cloud-Naive AI潜在瓶颈	eBPF的潜力
One-side RDMA	Agent/Tool冷启动	eBPF辅助Agent安全容器准确编排
内核空间RDMA	A2A/A2Tool第三方通信	eBPF辅助构建A2A/A2Tool共享内存地址空间 ²
构建全局内存池化	Agentic Workflow资源弹性	Agent/Tool拆分成小单元，eBPF辅助细粒度遥测与弹性伸缩 ³

1. Ideas源自：Coppock, Patrick H., et al. "LithOS: An operating system for efficient machine learning on GPUs." Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles. 2025.

2. Ideas源自：Lu, Fangming, et al. "Serialization/deserialization-free state transfer in serverless workflows." Proceedings of the Nineteenth European Conference on Computer Systems. 2024.

3. Ideas源自：Ruan, Zhenyuan, et al. "Quicksand: Harnessing Stranded Datacenter Resources with Granular Computing." 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25). 2025.

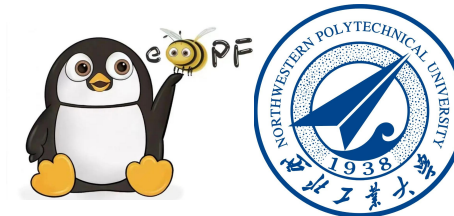


第四届 e B P F 开发者大会

The 4th eBPF Developer Conference (eBPFDC 2026)

附录 实验量化分析细节

陕西·西安 2026年4月18日



RNIC微架构资源解析

RNIC内部微架构组件



处理单元 (PU)
负责数据包处理



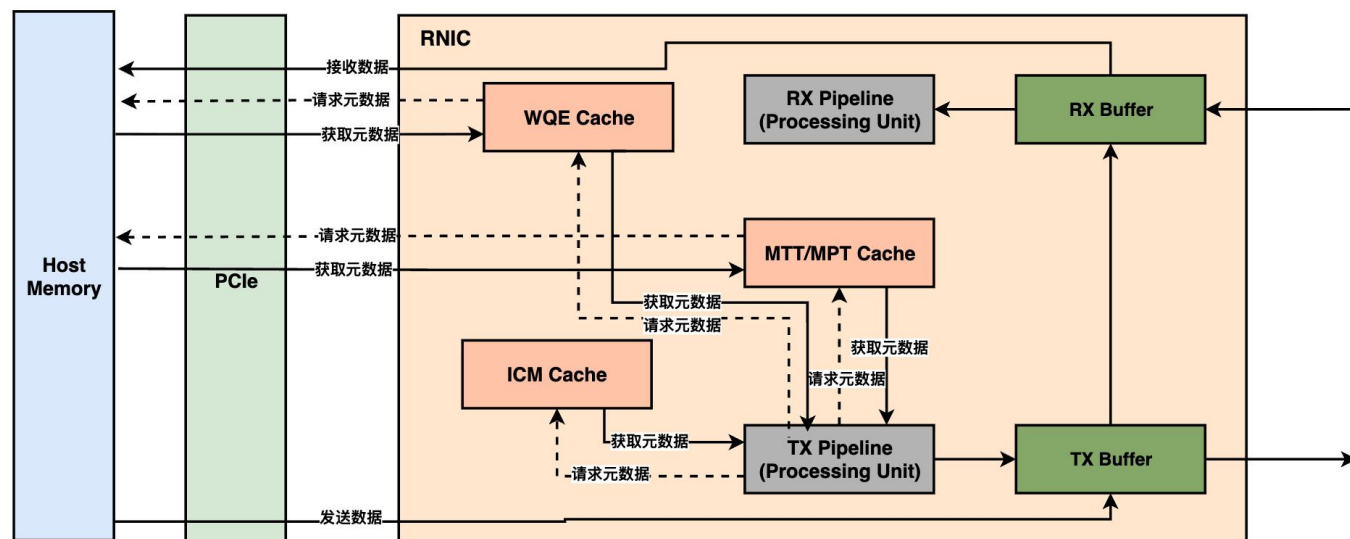
QP上下文缓存 (QPC)
存储连接状态



内存转换表缓存 (MTT)
VA→PA转换



工作队列条目缓存 (WQE)
存储工作队列条目



性能瓶颈来源

计算瓶颈

PU处理能力饱和

内存瓶颈

QPC/MTT缓存失效

总线瓶颈

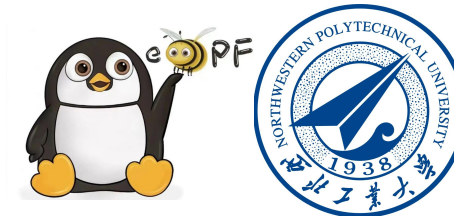
PCIe事务配额争用

实验环境

硬件: Mellanox ConnectX-6 Dx 100Gbps

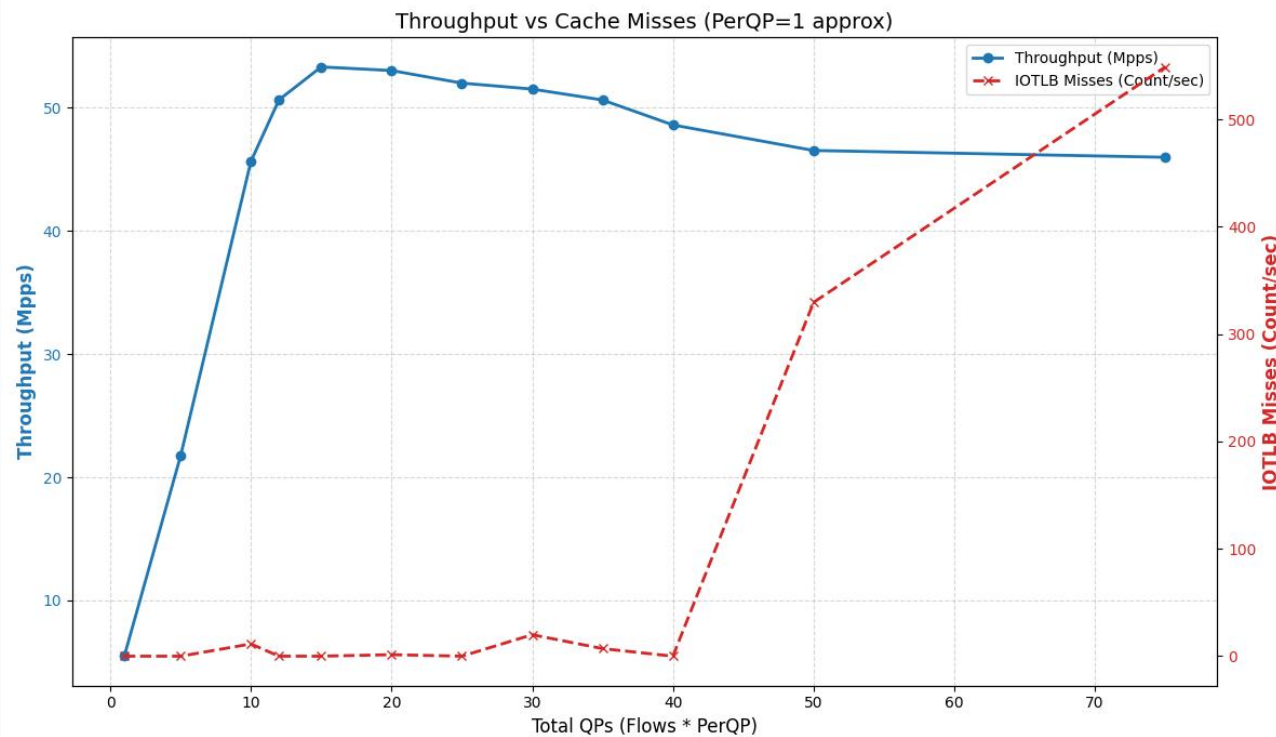
CPU: Intel Xeon Gold 5318Y

系统: Ubuntu 22.04, Linux 5.15



实验1：处理单元性能瓶颈实验与分析

QP数量 vs 包处理速率曲线



Cache 命中区
0-40 QP

性能峰值区
15-20 QP

性能稳态区
>50 QP

PU计算瓶颈

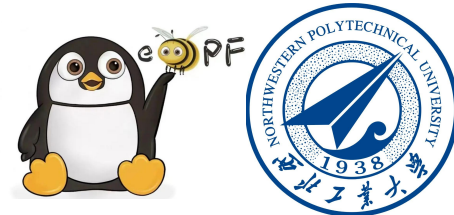
实验目标：隔离其他干扰，测定 PU 峰值包处理速率

测试方法：ib_write_bw 生成负载，Post-N批量提交多个 64B任务，增加总QP数，测量包处理速率

监控指标：RNIC包处理速率；PCIe链路(IOTLB Miss)

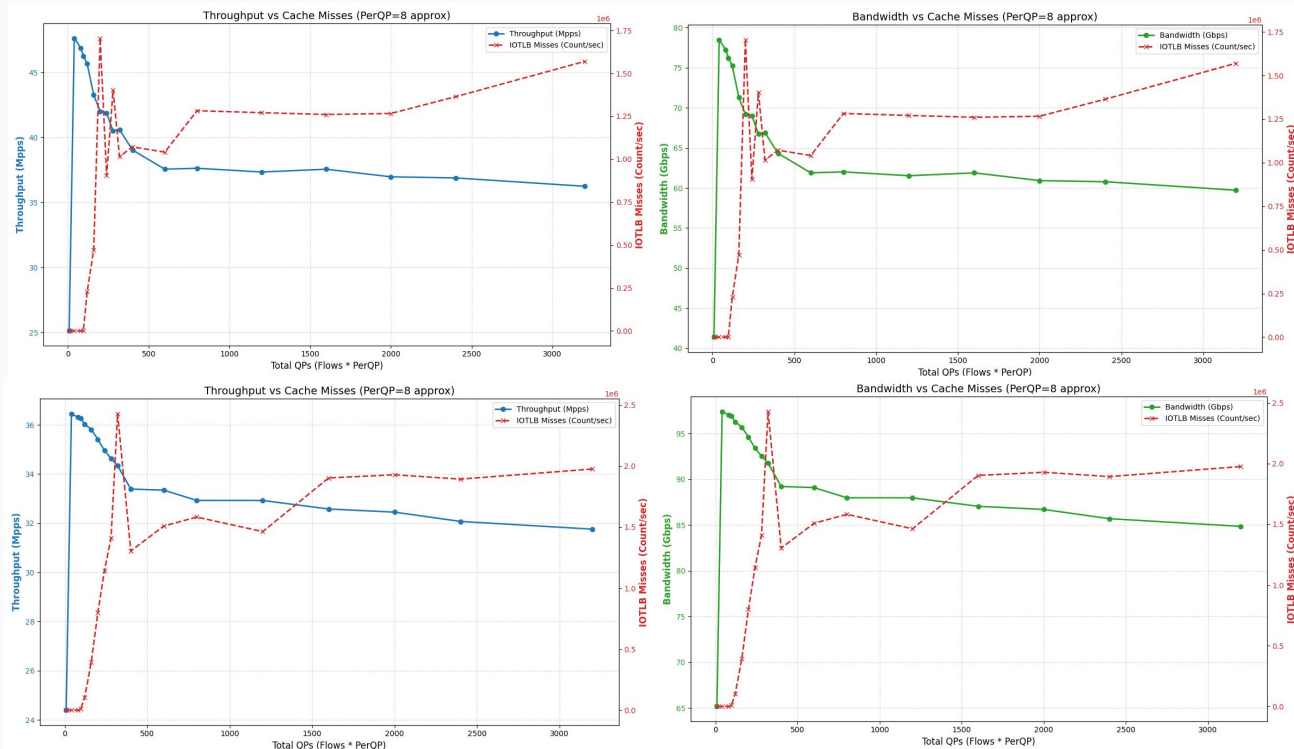
关键发现

- 1 处理能力：包处理速率峰值约53 Mpps
- 2 调度开销：Cache Miss前，性能略微下滑主导因素可能为PU上下文切换或调度开销



实验2.1: QP上下文缓存性能瓶颈实验 (多租户并发场景)

QP数量 vs 吞吐量性能曲线



Cache 命中区
0-80 QP

缓存抖动区
80-400 QP

性能稳态区
>400 QP

QP Cache瓶颈

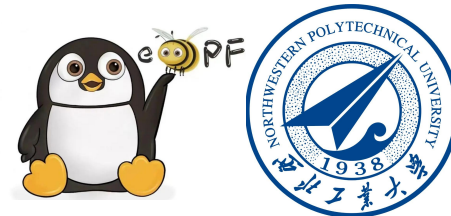
实验目标: 大规模连接QP 规模限制, 及其失效对网络性能的影响

测试方法: Post-8 模式, 多流 (多进程) 并发, 小 MR 以保 MTT 缓存命中。分别以128B 负载和256B负载测试。

监控指标: 带宽/包处理能力; PCIe链路 (IOTLB Miss)

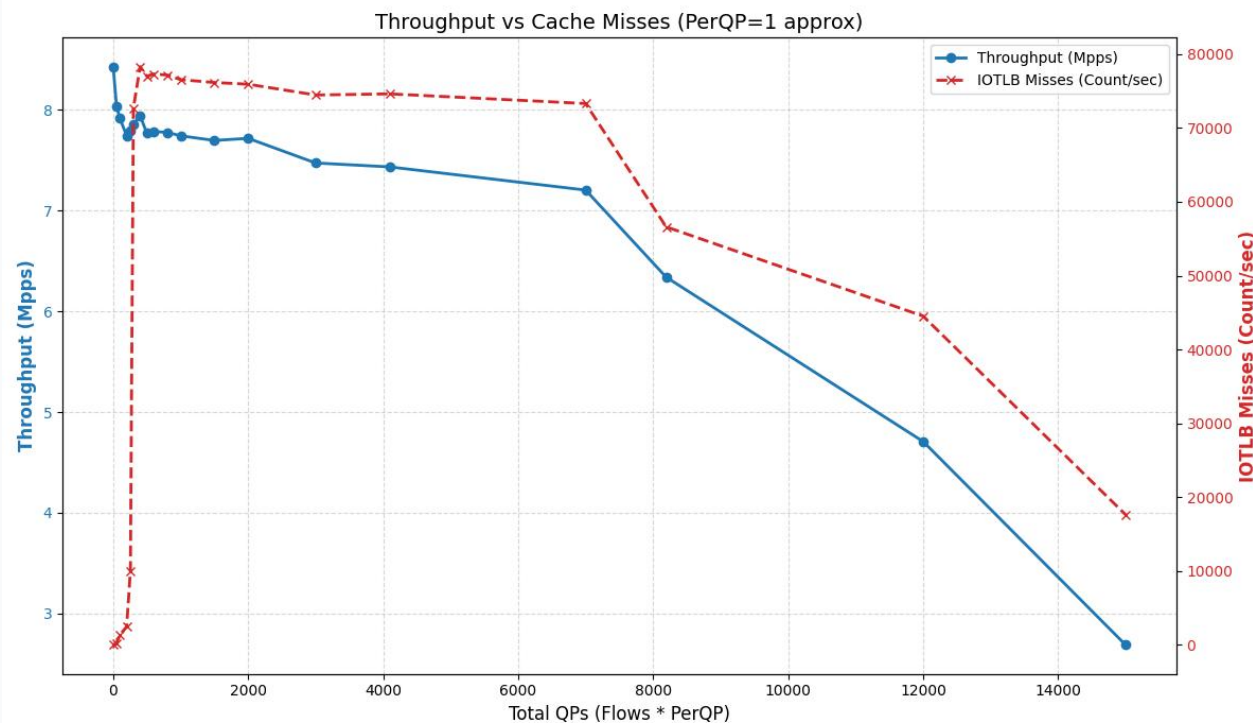
关键发现

1 系统级容量: 多租户场景下有效容量仅80 QP



实验2.2: QP上下文缓存性能瓶颈实验（单流隔离场景）

QP数量 vs 吞吐量性能曲线



Cache 命中区
0-256 QP

平稳区
300-4000 QP

崩溃点
>8192 QP

QP Cache瓶颈

实验目标: 单流连接下, RNIC 内部 QP 上下文缓存的容量限制, 及其失效对网络性能的影响

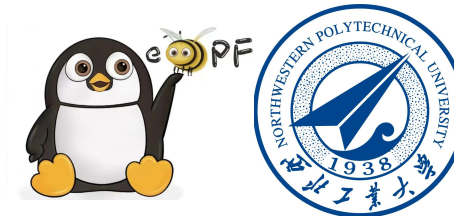
测试方法: 单流模式 (共享 PD 和页表以降低 IOTLB 压力), Post-1 模式 (最大化暴露缓存失效延迟惩罚)。

监控指标: 带宽/包处理能力; PCIe链路 (IOTLB Miss)

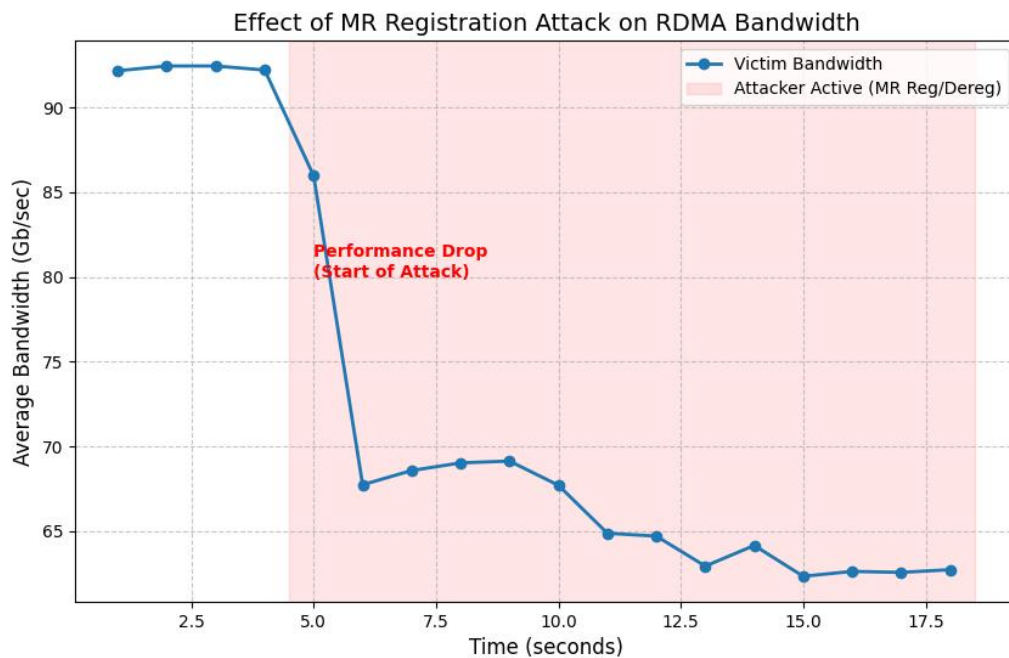
关键发现

- 1 有效容量为256 QP, 此时Miss 计数跃升
- 2 在 300 QP 时, Miss 瞬间暴涨 7 倍 (缓存完全溢出)
- 3 崩溃点: 超过8192 QP 时iotlb_miss 不升反降, 流水线严重阻塞导致总吞吐量骤降

实验3.1：地址转换表抖动验证实验



MTT抖动攻击实验结果



攻击前带宽

92 Gbps

攻击后带宽

60~65 Gbps

攻击机制

攻击方式

循环执行内存注册/销毁，不传输数据

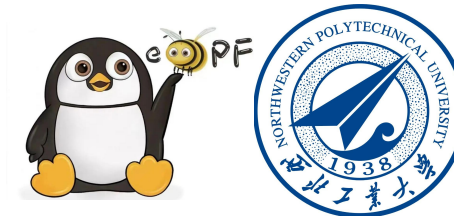
攻击效果

受害者带宽下降约30%

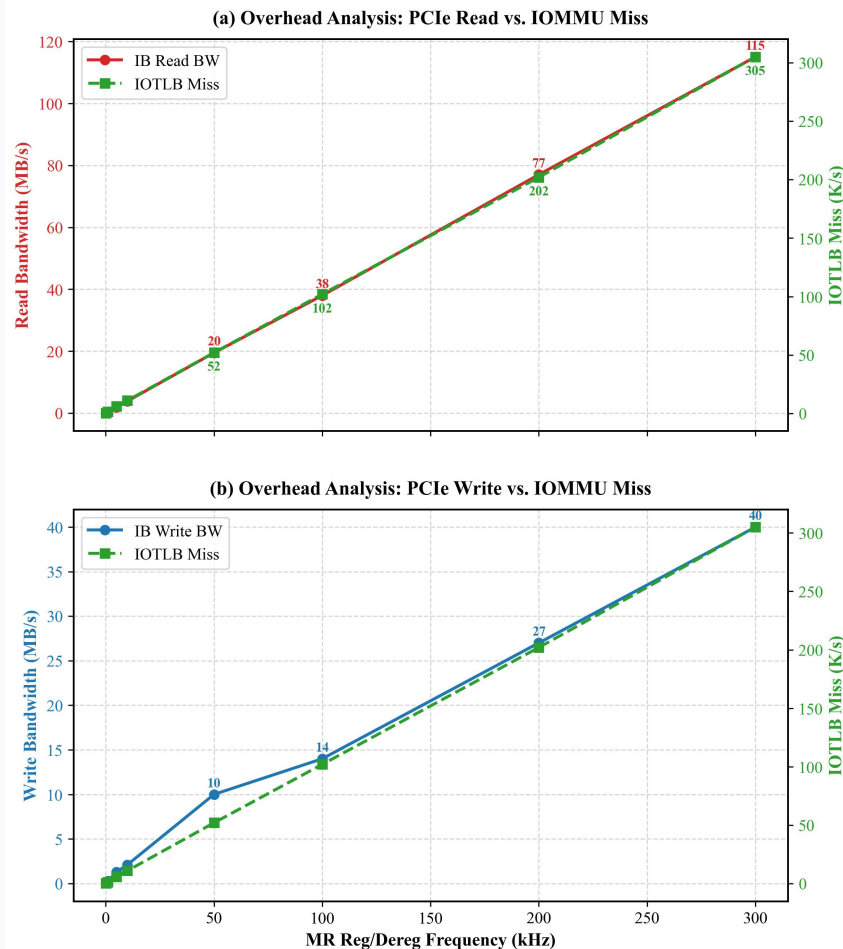
根本原因

高频操作破坏了片上缓存的时间局部性。MTT 重填作为高优先级非数据操作，抢占了 PCIe 事务队列，导致正常数据流水线严重阻塞

实验3.2：地址转换表抖动量化实验



MTT抖动量化实验结果



攻击方式

攻击方式

以指定频率（0-300kHz）执行内存注册/销毁，不传输数据

监控指标

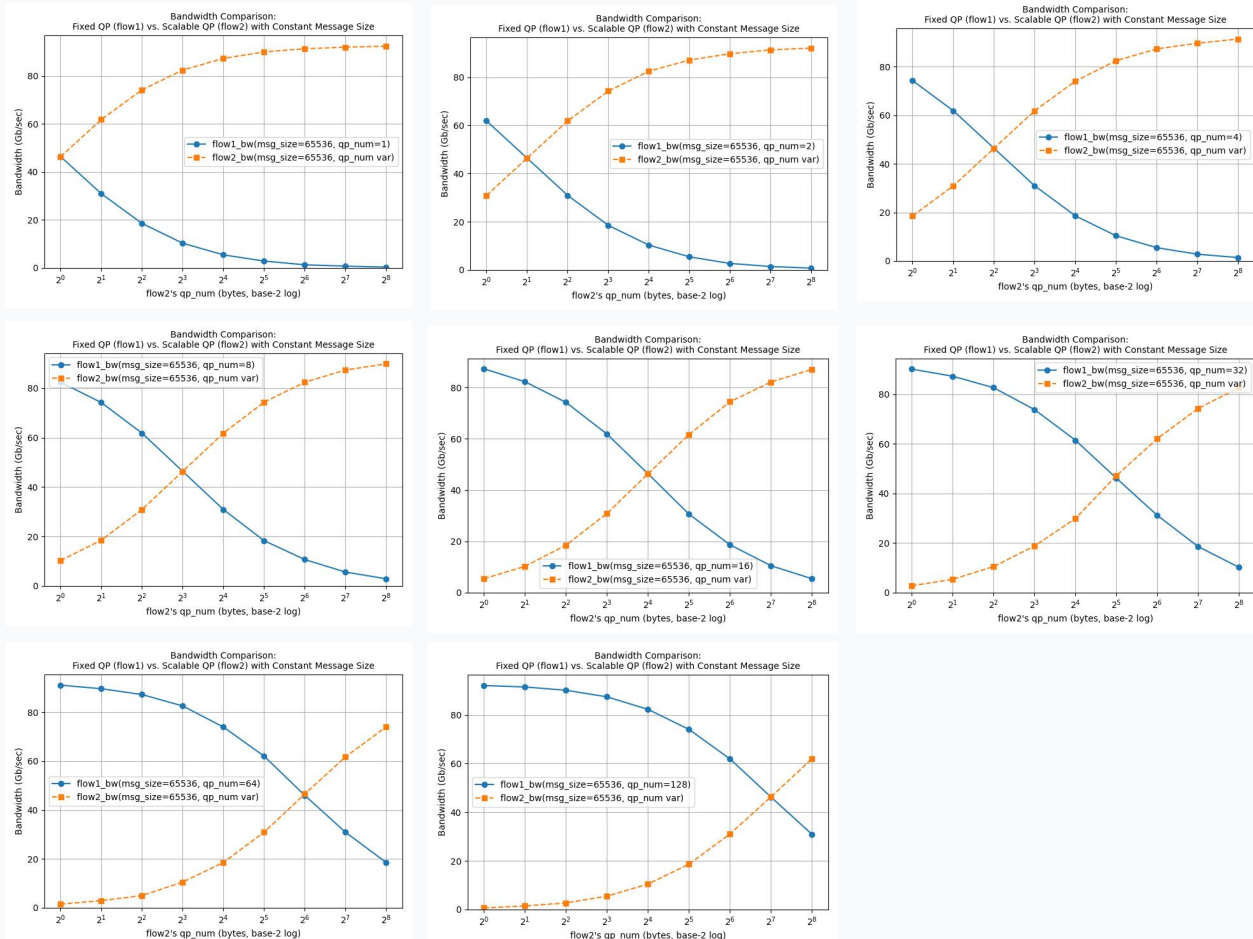
PCIe Read、PCIe Write 带宽；PCIe IOMMU Miss

关键发现

- 1 PCIe Read OPS 、 IOTLB Miss 均与攻击频率呈现近乎完美的线性正相关
- 2 单次操作触发6.2次 PCIe Read，这意味着一次 `ibv_reg_mr` 调用触发了 RNIC 发起约 6 次独立的 DMA 读取请求，用于从主机内存中拉取新的 MTT 条目及相关的 MPT 元数据。
- 3 单次操作触发1次 IOTLB Miss，说明每次新注册 MR 都会生成全新的虚拟地址映射

实验4: QP 数量对带宽分配的影响

调度探究实验



发现的问题

QP数量依赖

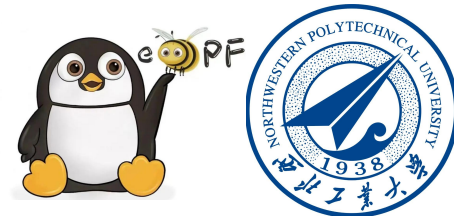
带宽分配 $\propto$ 活跃QP数量, 恶意用户可通过申请大量QP抢占带宽



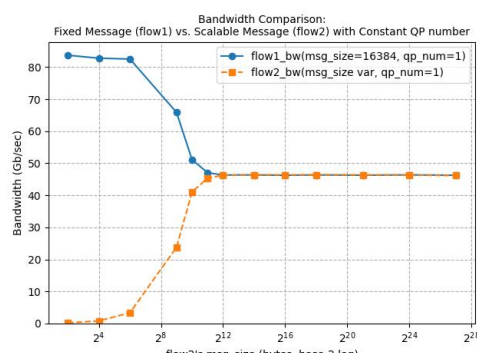
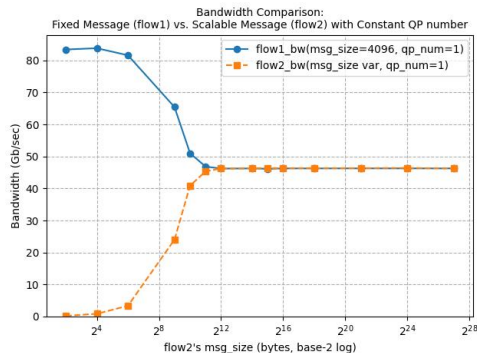
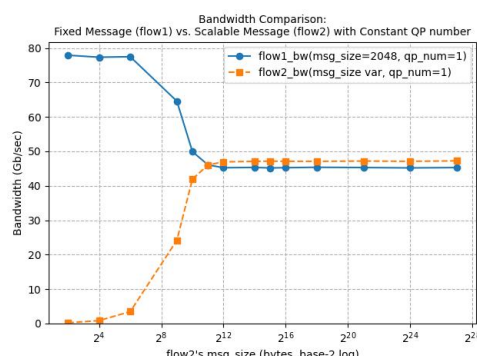
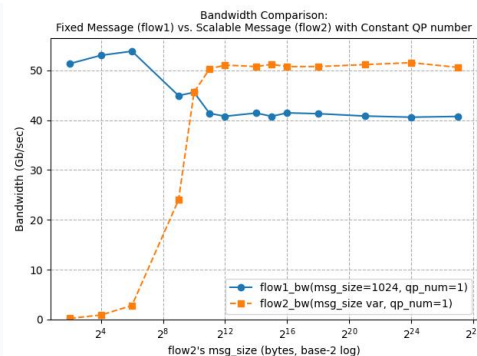
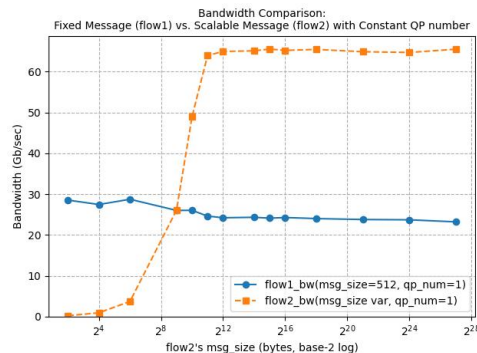
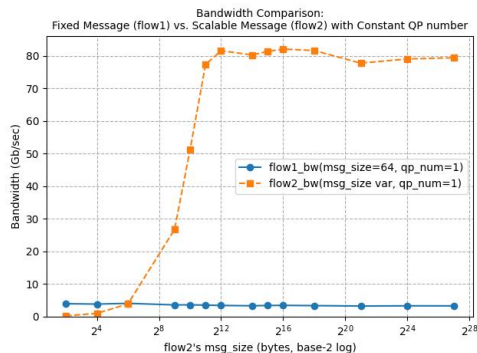
实验设置: 固定消息大小 (64KB), 两条流分配不同数量的 QP 竞争带宽。

实验结果: 带宽分配比例与 QP 数量呈近似正线性相关 (带宽比 $\approx$ QP 比)。

结论: RNIC 采用了基于 QP 的 Round-Robin (轮询) 调度机制



实验5：消息大小对带宽分配的影响



发现的问题

非面向字节调度

按“次”调度而非按“字节”，小消息流劣势

实验设置：固定单 QP，流1固定消息大小（64，512，1024，2048，4098，16384），流2在实验中变化消息大小（横坐标），MTU 设定为 4KB。

实验结果：**消息 > MTU**：带宽被平均分配（数据按 MTU 拆分为标准包轮询）；**消息 < MTU**：小消息流获得的有效带宽显著低于大消息流。

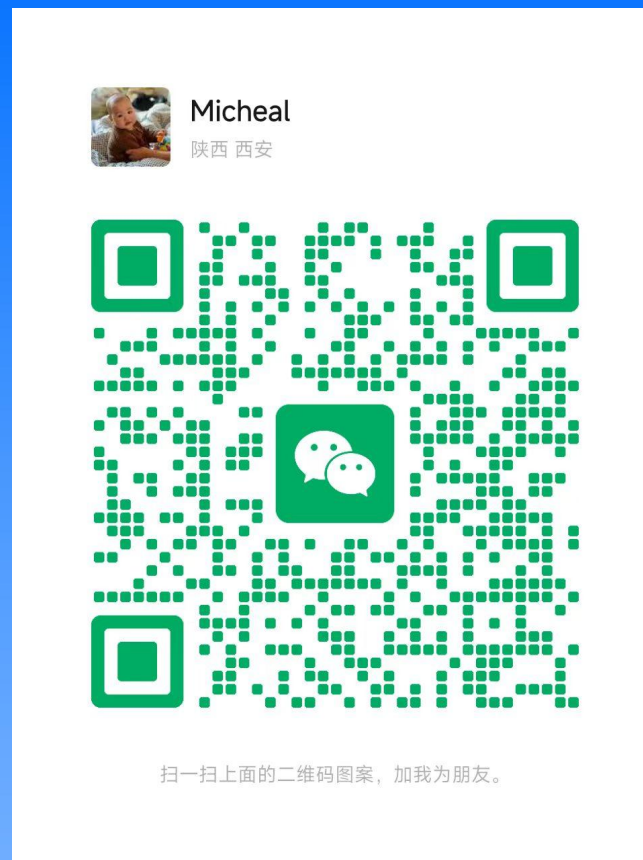
结论：消息被拆分为MTU大小的数据包，调度周期中每个QP只能发送一个数据包



第四届 eBPF 开发者大会

The 4th eBPF Developer Conference (eBPFDC 2026)

Thanks



陕西·西安 2026年4月18日